FORCER ESP32 DEVRE SİSTEMİ

Teknik Devir Teslim Raporu

v3.0 — Mart 2026

v2.0 üzerine: Ses Sistemi + Halka LED Senkronizasyon Oturumu

---

1. Bu Oturumun Kapsamı

Bu belge, v2.0 teknik devir teslim raporunun (FORCER_DEVIR_TESLIM.docx) üzerine eklenecek ek rapordur. v2.0'da 'Aktif Sorunlar' bölümünde listelenen sorunların çözüm süreci, bu oturumda (Mart 2026) Berk ile birlikte adım adım yürütülmüştür. Oturumda iki ana başlık ele alınmıştır:

- Bölüm 9'daki kritik sorun: WS2812B halka LED'in session durumuna göre güncellenememesi
- Ses sistemi (DFPlayer Mini) entegrasyonunun tamamlanması ve karşılaşılan sorunların giderilmesi

Bu oturumda hiçbir donanım değişikliği yapılmamıştır. Tüm müdahaleler yazılım düzeyindedir.

---

2. Başlangıç Durumu — v2.0 Aktif Sorunları

Oturum başında sistemin durumu şöyleydi:

Bileşen / Özellik	Durum
WiFi bağlantısı	Çalışıyor
4 buton GPIO okuma	Çalışıyor
3 küçük LED	Çalışıyor
Buzzer geri bildirimi	Çalışıyor
VPS Flask endpoint	Çalışıyor
Telethon daemon	Çalışıyor

WS2812B halka LED fiziksel bağlantı	Çalışıyor
WS2812B halka LED session senkronizasyonu	ÇALIŞMIYOR — Aktif sorun
DFPlayer Mini ses sistemi	Kısmen çalışıyor — Test edilmemişti
Alarm sistemi	Kodlandı, test edilmemişti

v2.0 raporunda sorunun muhtemel sebebi olarak 'last_status_check değişkeninin 0 kalması ve döngünün o bloğa ulaşamaması' gösterilmişti.

---

3. Sorun 1: Halka LED Session Senkronizasyonu

3.1 Gerçek Kök Neden Tespiti

v2.0 raporundaki teşhis kısmen doğruydu: last_status_check değişkeni gerçekten 0 kalıyordu. Ama bunun sebebi döngünün o bloğa ulaşamaması değildi. Asıl sebep şuydu:

> last_status_check = 0 olduğu için time.time() - 0 > 5 koşulu her iterasyonda true oluyor, yani update_leds() her 50ms'de çağrılıyordu. Ama update_leds() içinde urequests.get() bir MemoryError fırlatıyordu. MemoryError, Python'da BaseException'dan türüyor, Exception'dan değil. Mevcut except Exception as e: bloğu onu yakalayamıyor, hata kabarcıklanıyor, while True döngüsü çöküyor ve last_status_check = time.time() satırına hiç ulaşılamıyordu.

Bunu doğrulamak için Thonny Shell'den şu kontrol yapıldı:

```
print(last_status_check)    # Çıktı: 0
print(time.time())          # Çıktı: 828017733
print(time.time() - last_status_check)   # Çıktı: 828017733
```

Bu çıktı, döngünün bloğa ulaşmadığını değil, ulaştığı ama update_leds() içinde crash oluştuğunu ve last_status_check'in hiç güncellenmediğini gösteriyordu. 'Saatler sonra kendiliğinden düzeliyor' davranışı da bu açıklamayla tutarlıydı: MicroPython'un garbage collector zaman zaman belleği temizleyince MemoryError geçici olarak ortadan kalkıyordu.

3.2 Uygulanan Çözüm

İki kod değişikliği yapıldı:

Değişiklik 1 — update_leds() içinde except BaseException

Eski kod:

```
except Exception as e:
    print(f'VPS baglanti hatasi: {e}')
```

Yeni kod:

```
except BaseException as e:
```

```
        print(f'UPDATE_LEDS HATA TIPI: {type(e).__name__}: {e}')
```

Bu değişiklik hem MemoryError'ı yakalamayı hem de hata tipini loglayarak teşhisi kolaylaştırmayı sağladı.

Değişiklik 2 — Ana döngüde try/finally sarmalama

Eski kod:

```
    if (time.time() - last_status_check) > 5:
        update_leds()
        last_status_check = time.time()
```

Yeni kod:

```
    if (time.time() - last_status_check) > 5:
        try:
            update_leds()
        except BaseException as e:
            print(f'DONGÜ CRASH: {type(e).__name__}: {e}')
        finally:
            last_status_check = time.time()  # crash olsa bile
    güncellenir
```

finally bloğu sayesinde update_leds() crash olsa bile last_status_check güncelleniyor ve döngü sonsuz crash döngüsüne girmiyordu.

3.3 Halka LED Senkronizasyonunun Doğrulanması

Düzeltme sonrasında Shell'den şu testler yapıldı:

```
    update_leds()
    print(ring_durum)     # Çıktı: in_session
    print(last_known_durum)  # Çıktı: in_session
```

> Manuel çağrımda ring_durum doğru güncelleniyor. Halka LED doğru rengi gösteriyor.

Ancak döngüde otomatik çalışırken halka kırmızı kalmaya devam etti. Ek debug ile update_ring() içinde update_leds() işlevi beklenen şekilde çalışıyordu. Sorun, led_locked_until koşulunun bazen set_ring() çağrısını engellemesiydi.

3.4 led_locked_until Koşulunun Kaldırılması

update_leds() içindeki orijinal kod:

```
    if time.time() > led_locked_until:
        led_red.value(0); led_yellow.value(0); led_green.value(0)
        if last_known_durum == 'in_session': led_green.value(1)
        elif last_known_durum == 'on_break': led_yellow.value(1)
        else: led_red.value(1)
        set_ring(last_known_durum)
```

Bu koşul, buton basışından 12 saniye sonrasına kadar VPS güncellemesinin LED'leri değiştirmesini engelliyordu. Buton basışında set_led() zaten doğru rengi anlık ayarlıyordu, dolayısıyla bu koruma gereksizdi ve set_ring()'in çağrılmasını engelliyordu.

Yeni kod — koşul tamamen kaldırıldı:

```
led_red.value(0); led_yellow.value(0); led_green.value(0)
if last_known_durum == 'in_session': led_green.value(1)
elif last_known_durum == 'on_break': led_yellow.value(1)
else: led_red.value(1)
set_ring(last_known_durum)
```

3.5 Thonny — ESP32 Boot Sorunu

Çözüm sonrasında test sırasında beklenmedik bir sorunla karşılaşıldı: Thonny bağlıyken ESP32 adaptörle çalışmıyordu, main.py flash'a kaydedilmemişti.

Sorunun anatomisi:

- Thonny'de 'Run current file' butonu ESP32'ye dosyayı kaydetmez, sadece çalıştırır.
- Adaptöre takılınca flash'taki main.py çalışır. Flash boşsa hiçbir şey çalışmaz.
- Thonny bağlıyken soft reset atar ve main.py'yi yeniden başlatır, ardından kontrolü bırakmaz. Bu yüzden döngü başlamıyor gibi görünüyordu.
- boot.py içine kod kaydedilmişti. boot.py'deki girintili while True bloğu bozuktu.

Kalıcı çözüm iş akışı:

- Kodu düzenle, File → Save as → MicroPython device → main.py olarak kaydet.
- boot.py içine sadece 'pass' yaz, MicroPython device'a kaydet.
- Thonny'yi kapat.
- ESP32'yi adaptöre tak — otomatik başlar.
- Debug gerekirse USB'ye tak, Thonny'yi aç, Ctrl+C ile durdur, Shell'den çalış.

> Halka LED sorunu bu oturumda tamamen çözüldü. Sistem session durumuna göre doğru rengi gösteriyor.

---

4. Sorun 2: DFPlayer Mini Ses Sistemi

4.1 İlk Durum

v2.0 raporunda 'DFPlayer ses sistemi tam test edilmedi' olarak işaretlenmişti. Bu oturumda ses sistemi ilk kez kapsamlı şekilde test edildi ve birden fazla sorunla karşılaşıldı.

4.2 Sorun: Ses Grupları Arası Sarkma (Auto-Advance)

DFPlayer Mini bir ses dosyasını çaldıktan sonra otomatik olarak SD karttaki sıradaki dosyaya geçiyor. Bu davranış şu soruna yol açıyordu:

- Mola sesi (0005.mp3) bittikten sonra DFPlayer otomatik olarak 0006.mp3'ü (hgb_1) çalıyordu.
- Hgb sesi (0010.mp3) bittikten sonra 0011.mp3 (devam_1) çalıyordu.
- Kullanıcı tarafından 'hgb butonunda mola sesi çalıyor' olarak algılanıyordu.

Denenen ve başarısız olan çözümler:

- dfplayer_cmd(0x08, 0, 0x00) komutu — yanlış komuttu, sistemi bozdu.
- SD karta boş (sessiz) mp3 ekleme — boş ses çalınıyordu, kullanıcı deneyimi kötüleşti.
- play() içinde stop() + time.sleep(0.3) kombinasyonu — sesleri kesti ama sıralamayı bozmadı.

Kalıcı çözüm — play() fonksiyonuna ses bittikten 30 saniye sonra otomatik durdurma eklendi:

```
def play(dosya_no):
    global play_token
    play_token += 1
    token = play_token
    def _play():
        time.sleep(1.5)
        if play_token == token:
            dfplayer_cmd(0x03, 0, dosya_no)
            time.sleep(30)
            if play_token == token:
                stop()
    _thread.start_new_thread(_play, ())
```

Ses dosyaları 30 saniyeden kısa olduğundan, 30 saniye sonra stop() gönderilmesi auto-advance'e fırsat bırakmaz.

4.3 Sorun: Thread Birikimiyle Yanlış Ses Çalınması

Orijinal play() fonksiyonu her çağrıda yeni bir thread açıyor ve 1.5 saniye bekleyip ses çalıyordu. Bu yapıda iptal mekanizması yoktu:

- Molaya basıldı → thread A açıldı, 1.5 sn bekliyor.
- 1 saniye sonra hgb'ye basıldı → thread B açıldı.
- 0.5 saniye sonra thread A uyanıp mola sesini çaldı.
- Ardından thread B uyanıp hgb sesini çaldı.
- Kullanıcı tarafından 'rasgele ses çalıyor' olarak algılanıyordu.

Çözüm — play_token sistemi:

Her yeni butona basışta play_token global değişkeni artırılır. Thread uyandığında token'ı kontrol eder — eğer token değişmişse (yani yeni butona basılmışsa) komutu göndermez. Bu sayede yalnızca en son istenen ses çalınır.

```
play_token = 0

def play(dosya_no):
    global play_token
    play_token += 1
    token = play_token
    def _play():
        time.sleep(1.5)
        if play_token == token:    # Eski thread iptali burada
            dfplayer_cmd(0x03, 0, dosya_no)
            time.sleep(30)
            if play_token == token:
                stop()
    _thread.start_new_thread(_play, ())
```

4.4 Sorun: devam Butonunda stop() Çağrısı

Orijinal set_led() içinde yalnızca 'devam' komutunda stop() çağrısı vardı, diğerlerinde yoktu. Bu tutarsızlık seslerin yarıda kesilmesine neden oluyordu. stop() satırı kaldırıldı; artık tüm butonlar için davranış tutarlı.

4.5 SD Kart Yeniden Yapılandırma

Bu oturum boyunca SD kart birden fazla kez yeniden düzenlendi. Her düzenlemede DFPlayer'ın dahili indeks tablosu bozuldu — bu da seslerin yanlış sırada çalınmasına neden oldu.

> DFPlayer Mini, dosyaları SD karta kopyalanma sırasına göre indeksler — dosya adına göre değil. FAT32 tablosundaki kayıt sırası belirleyicidir. Toplu kopyalama yanlış sıra oluşturabilir.

Doğru SD kart hazırlama prosedürü:

- SD kartı FAT32 olarak tam format at (hızlı format değil).
- mp3/ klasörü oluştur.
- Dosyaları tek tek, sırayla kopyala: önce 0001.mp3, bitince 0002.mp3, sırasıyla devam et.
- Toplu sürükleme yapma — kopyalama sırası garanti edilmez.

Final SD kart yapısı (24 dosya):

Dosya	İçerik
0001.mp3	mola_1
0002.mp3	mola_2
0003.mp3	mola_3
0004.mp3	mola_4
0005.mp3	mola_5

0006.mp3	hgb_1
0007.mp3	hgb_2
0008.mp3	hgb_3
0009.mp3	hgb_4
0010.mp3	hgb_5
0011.mp3	devam_1
0012.mp3	devam_2
0013.mp3	devam_3
0014.mp3	devam_4
0015.mp3	devam_5
0016.mp3	bitir_1
0017.mp3	bitir_2
0018.mp3	bitir_3
0019.mp3	bitir_4
0020.mp3	bitir_5
0021.mp3	gun_sonu_1
0022.mp3	gun_sonu_2
0023.mp3	gun_sonu_3
0024.mp3	alarm (şu an silinmiş, yer tutucu)

4.6 Alarm Sistemi — Devre Dışı Bırakıldı

Orijinal kodda alarm sistemi (check_alarm, play_alarm) mevcuttu. Bu oturumda molaya basıldığında alarm sesinin (0024.mp3) yanlış tetiklendiği tespit edildi. Sorunun kök nedeni şuydu:

- mola_baslangic = time.time() set ediliyor.
- NTP sync öncesinde time.time() küçük bir değer döner (boot'tan saniye sayar).
- NTP sync sonrasında time.time() büyük bir Unix timestamp'e atlar.
- Bu durumda mola_sure = time.time() - mola_baslangic aniden milyonlarca saniye oluyor.
- Alarm koşulu (mola_sure >= 15*60) hemen tetikleniyor.

Çözüm denemeleri başarısız oldu. Alarm sistemi tamamen devre dışı bırakıldı:

```
def check_alarm():
    pass
```

Alarm sistemi ileride NTP sync garantisi sağlandıktan sonra yeniden entegre edilebilir.

---

5. Donanım Bulguları

5.1 GPIO34 — PULL_DOWN Uyumsuzluğu

ESP32'de GPIO34-39 aralığındaki pinler input-only'dir ve dahili pull-up/pull-down direnci desteklemez. Orijinal kodda şu tanım mevcuttu:

```
'bitir': machine.Pin(34, machine.Pin.IN, machine.Pin.PULL_DOWN)
```

Bu tanım boot sırasında şu hatayı üretiyordu:

```
E (1099) gpio: gpio_pulldown_en(116): GPIO number error (input-only
pad has no internal PD)
```

Önceki oturumlarda çalışıyordu çünkü bu hata fatal değil, warning seviyesinde — ESP32 devam ediyordu. Ancak pin floating kalabilir ve hayalet tetiklemelere neden olabilir.

Düzeltme:

```
'bitir': machine.Pin(34, machine.Pin.IN)   # PULL_DOWN kaldırıldı
```

Breadboard'daki 10kΩ pull-down direnci zaten bu görevi yapıyor.

5.2 WiFi Bağlantı Kopması

Router, ESP32'yi uzun süre idle görünce DHCP lease'ini düşürüyordu. Orijinal kodda 50 deneme (10 saniye) vardı. Bu yetersiz kaldı.

Çözüm — deneme sayısı 50'den 150'ye çıkarıldı (30 saniye):

```
for _ in range(150):
```

Ek öneri (henüz uygulanmadı): Router'da ESP32'nin MAC adresine sabit IP tanımlanması. Bu, DHCP lease sorununu tamamen ortadan kaldırır.

5.3 update_ring() — Crash Tespiti

Ana döngüde update_ring() try/except olmadan çağrılıyordu. np.write() zaman zaman exception fırlatabilir. Bu crash'i yakalamak ve last_ring_tick'in güncellenmesini garanti altına almak için sarmalandı:

```
if time.ticks_diff(now_ms, last_ring_tick) >= 500:
    try:
        update_ring()
    except BaseException as e:
        print(f'UPDATE_RING CRASH: {type(e).__name__}: {e}')
    last_ring_tick = now_ms  # crash olsa bile güncellenir
```

---

6. Yeni Özellikler — Bu Oturumda Eklendi

6.1 PARLAKLIK_YESIL Değişkeni

Yeşil LED'in (in_session durumu) parlaklığı diğer renklerden ayrı ayarlanabilir hale getirildi. Kodun başına eklendi:

```
PARLAKLIK = 40
PARLAKLIK_YESIL = 20   # Yeşil için ayrı parlaklik
```

update_ring() içindeki in_session dalı güncellendi:

```
if ring_durum == 'in_session':
    ring(0, PARLAKLIK_YESIL, 0)
```

6.2 ara Session Durumu

Forcer bot tarafında yeni bir session türü eklendi: 'ara'. Bu durum hgb'den farklıdır:

Özellik	Değer
Kullanım amacı	Uzun mecburi ara (yemek, dışarı çıkma, aile vb.)
Süre sınırı	Yok
Alarm	Yok
Mola sayısı etkisi	Artmaz
Odak etkisi	Etkilenmez
Süre	Durur (dondurulur)
Geri dönüş	devam / döndüm / geldim komutu

hgb ile farkı: hgb maksimum 10 dakika ve hatırlatma cron'u var. ara'nın süre sınırı yok.

Fiziksel buton yoktur — yalnızca Telegram üzerinden Forcer'a yazılarak tetiklenir.

Halka LED davranışı: İlk 4 LED sarı, son 4 LED kırmızı (sürekli, blink değil).

update_ring() içine eklenen kod:

```
elif ring_durum == 'ara':
    for i in range(4):
        np[i] = (PARLAKLIK, PARLAKLIK, 0)   # sarı
    for i in range(4, 8):
        np[i] = (PARLAKLIK, 0, 0)           # kırmızı
    np.write()
```

6.3 hgb ve ara için update_leds() Güncellemesi

update_leds() içindeki küçük LED yönetimi güncellendi. hgb ve ara session'larında küçük LED sarı yanar:

```
if last_known_durum == 'in_session':
    led_green.value(1)
elif last_known_durum == 'on_break':
    led_yellow.value(1)
elif last_known_durum == 'hgb':
```

```
        led_yellow.value(1)
    elif last_known_durum == 'ara':
        led_yellow.value(1)
    else:
        led_red.value(1)
```

---

7. Session Durumu — LED Eşleşme Tablosu (Güncel)

Session Durumu	Küçük LED	Halka LED
in_session	Yeşil	Yeşil sürekli (PARLAKLIK_YESIL=20)
on_break	Sarı	Sarı sürekli
idle	Kırmızı	Kırmızı yanıp söner (500ms)
day_closed	Kırmızı	Kırmızı sürekli
hgb	Sarı	Sarı yanıp söner (500ms)
ara	Sarı	4 sarı + 4 kırmızı sürekli
wifi_error	Üçü birden	Beyaz yanıp söner (500ms)
deep sleep	Hepsi kapalı	Kapalı

---

8. Ses Komutu Eşleşme Tablosu (Güncel)

Değişken	SD Kart Dosyaları
SES_MOLA	0001–0005 (5 dosya, rastgele)
SES_HGB	0006–0010 (5 dosya, rastgele)
SES_DEVAM	0011–0015 (5 dosya, rastgele)
SES_BITIR	0016–0020 (5 dosya, rastgele)
SES_GUN_SONU	0021–0023 (3 dosya, rastgele)

Not: Alarm sesi (0024.mp3) ve alarm sistemi bu oturumda devre dışı bırakıldı. SD kartta yer tutucu olarak kalabilir veya silinebilir.

---

9. Final main.py Kod Değişiklikleri Özeti

Aşağıdaki tablo, orijinal koddan (thonny_last_x.txt) bu oturum sonundaki final koda kadar yapılan tüm değişiklikleri özetlemektedir:

Konum	Değişiklik	Sebep
pins tanımı — bitir	PULL_DOWN kaldırıldı	GPIO34 dahili pull-down desteklemez
SES_ALARM = 24	Satır kaldırıldı	Alarm devre dışı
play_alarm() fonksiyonu	Fonksiyon kaldırıldı	Alarm devre dışı
check_alarm() fonksiyonu	İçi boşaltıldı (pass)	Alarm devre dışı
play() fonksiyonu	play_token sistemi + 30sn stop eklendi	Thread birikimiyle yanlış ses sorunu
set_led('devam')	stop() satırı kaldırıldı	Tutarsız ses kesimi
update_leds()	led_locked_until koşulu kaldırıldı	set_ring() çağrısını engelliyordu
update_leds()	except BaseException eklendi	MemoryError yakalanmıyordu
update_leds()	hgb ve ara için sarı LED eklendi	Yeni session durumları
Ana döngü — update_leds()	try/finally ile sarmalandı	last_status_check her zaman güncellenir
Ana döngü — update_ring()	try/except ile sarmalandı	np.write() crash'ini yakalar
update_ring()	PARLAKLIK_YESIL kullanımı	Yeşil parlaklık ayrıştırıldı
update_ring()	ara durumu eklendi	Yeni session türü
connect_wifi()	50 → 150 deneme	Router DHCP drop sorunu
PARLAKLIK_YESIL = 20	Yeni sabit eklendi	Yeşil LED parlaklık kontrolü
play_token = 0	Yeni global değişken	Thread iptal sistemi

---

10. Açık Sorunlar ve Sonraki Adımlar

Öncelik	Görev	Notlar
□ Devam eden	Ses yarıda kesilme — parazit şüphesi	Temas sorunu olabilir, DFPlayer VCC/GND kontrol edilmeli
□ Yüksek	Alarm sistemini yeniden entegre et	NTP sync garantisi sağlandıktan sonra
□ Yüksek	Sarı buton değiştir	16mm panel buton, mekanik arıza
□ Yüksek	Deep sleep fiziksel test	Kodlandı, henüz test edilmedi
□ Orta	Flask ve daemon systemd servisi	VPS restart sonrası manuel başlatma gerekiyor
□ Orta	Router'da ESP32 MAC'e sabit IP	WiFi kopma sorununu kalıcı çözer

□ Orta	'bitir' on_break'de oturumu kapatmıyor	Forcer tarafında kural güncellemesi gerekiyor
□ Düşük	Flask endpoint token güvenliği	Açık internet erişimi var
□ Düşük	3D baskı kutu	OpenSCAD kodu hazır, baskı yapılmadı
□ Düşük	Isıyla daralan makaron	Buton kabloları için

---

11. VPS Altyapısı — Değişmeyen Bilgiler

Parametre	Değer
VPS IP	204.168.129.85
Flask port	5000
Hosting	Hetzner Cloud
OS	Ubuntu 24, Python 3.12
status_server.py	/home/openclaw/status_server.py
telegram_daemon.py	/home/openclaw/telegram_daemon.py
MEMORY.md	/home/openclaw/.openclaw/workspace-koc/MEMORY.md
Forcer bot	@berkinakillikocubot
Telegram Chat ID (Berk)	6288779679
openclaw agent	koc

Servis başlatma komutları (değişmedi):

```
pkill -f status_server.py
nohup python3 ~/status_server.py > ~/status_server.log 2>&1 &
pkill -f telegram_daemon.py
nohup python3 ~/telegram_daemon.py >> ~/telegram_daemon.log 2>&1 &
curl http://204.168.129.85:5000/status
```

---

12. Tamamlanan ve Çalışan Kısımlar (Güncel)

Özellik	Durum
ESP32 + MicroPython kurulumu	✅Çalışıyor
WiFi bağlantısı ve yeniden bağlanma	✅Çalışıyor (30sn timeout)
4 buton GPIO okuma	✅Çalışıyor
3 küçük LED durum göstergesi	✅Çalışıyor

Buzzer geri bildirimi	✅Çalışıyor
VPS Flask endpoint	✅Çalışıyor
Telethon daemon	✅Çalışıyor
Forcer komut entegrasyonu	✅Çalışıyor
WS2812B halka LED session senkronizasyonu	✅Çözüldü (bu oturumda)
WS2812B ara session rengi (4 sarı + 4 kırmızı)	✅Eklendi (bu oturumda)
DFPlayer Mini bağlantısı	✅Çalışıyor
Ses dosyaları SD karta yüklendi	✅Tamam (24 dosya, sıralı)
play_token thread iptal sistemi	✅Eklendi (bu oturumda)
NTP saat senkronizasyonu	✅Çalışıyor
Deep sleep (day_closed'da)	✅Kodlandı, fiziksel test yapılmadı
Alarm sistemi	□ Devre dışı bırakıldı
Ses yarıda kesilme	⚠ Devam eden sorun — parazit şüphesi
3D baskı kutu	✅Kod hazır, baskı yapılmadı